
Reducing Hallucinations in Structured Workflow Generation via Retrieval-Augmented Generation(RAG)

Haonan Gao
Yale University
eric.gao@yale.edu

Xuanyou Li
Yale University
xuanyou.li@yale.edu

Abstract

Large language models (LLMs) can translate plain-English instructions into JSON workflows that automate business processes, yet they occasionally *hallucinate* nonexistent step names or table references. We investigate a lightweight retrieval-augmented generation (RAG) pipeline that supplies a shortlist of valid items at inference time, thereby grounding the LLM without model fine-tuning. On a 300-example synthetic benchmark, a MiniLM retriever paired with GPT-4o-mini eliminates all table hallucinations, raises trigger accuracy from 77 % to 100 %, and cuts step hallucinations by one-third. The main trade-off is lower step recall when the retriever misses a gold item. We analyze this precision–recall tension and outline avenues for improving retrieval coverage.

1 Introduction

LLMs routinely convert natural language into structured artifacts (SQL, code, JSON workflows), enabling no-code automation platforms. Reliability remains a concern: models may *hallucinate* step names or table identifiers that do not exist, breaking execution and eroding trust. Fine-tuning an LLM to every enterprise schema is costly.

We explore a cheaper alternative: *retrieval-augmented generation* (RAG). At inference time we retrieve valid step/table names and prepend them to the prompt so the LLM can copy from a safe vocabulary. Our study reproduces this idea at small-scale and quantifies the precision–recall trade-off introduced by imperfect retrieval.

2 Related Work

Hallucination mitigation spans constrained decoding, post-hoc validation, and prompt design [1, 2]. RAG grounds LLMs in retrieved evidence, boosting factual accuracy in QA and dialogue [4]. ServiceNow researchers recently applied RAG to internal workflow JSONs and reduced hallucinated steps by 70 %. We provide a reproducible, low-resource replica and dissect failure modes.

3 Method

3.1 Mathematical Setting

Let q denote a natural-language requirement. The gold output is a JSON pair $y = (\tau, S)$ where $\tau = \{\text{type}, \text{table}\}$ is the trigger and $S = \langle s_1, \dots, s_m \rangle$ is an ordered list of step names from a lexicon $\mathcal{V}_{\text{step}}$. We define a retriever $R: q \mapsto C \subset \mathcal{V}_{\text{step}} \times \mathcal{V}_{\text{tbl}}$ that returns the top- k candidate step/table names, and a generator G (the LLM) that maps $(q, C) \mapsto \hat{y}$. Without retrieval (BASE) we pass $C = \emptyset$.

```

{
  "trigger": {"type": "on_insert", "table": "user"},
  "steps": [
    {"id": "1", "name": "send_email", "parent": null},
    {"id": "2", "name": "archive_record", "parent": "1"}
  ]
}

```

Figure 1: Target JSON structure.

The objective is to maximize task accuracy while minimizing hallucinations: $\hat{y}$ should contain only elements from gold y .

3.2 Dataset

We synthesize 300 (q, y) pairs. Step pool: 12 verbs {`create_ticket`, `post_to_slack`, ...}. Table pool: 5 tables {`incident`, `change_request`, ...}. Each query describes a trigger event and 2–4 steps (e.g. “*When a new user is created, send an email and archive the record.*”). JSONs follow the schema in Fig. 1. Splits: 200/50/50 train-dev-test. Code for generation and JSON templates is released.¹

3.3 Retrievers

TF-IDF We vectorize each step/table name using unigrams and compute cosine similarity to the query.

MiniLM We encode query and candidate names with `paraphrase-MiniLM-L6-v2` (33M params) and index vectors with FAISS (inner-product). *Fine-tuning.* Positive pairs are (q, s_i) for every gold step s_i and (q, τ_{table}) . We sample three random negatives per positive and optimize a contrastive loss for 5 epochs.

3.4 Generation Model and Prompting

We use **GPT-4o-mini** via OpenAI API. Temperature 0 for deterministic output. BASE prompt: requirement q + schema. RAG prompt: “*Available steps: C_{step} ; tables: C_{tbl} ... JSON:*”

3.5 Computing Infrastructure

Training and inference ran on Google Colab:

- **GPU:** 1 NVIDIA T4 (16GB) for MiniLM fine-tuning.
- **CPU:** Intel Xeon host for FAISS retrieval and GPT API calls.
- **Software:** Ubuntu 22.04, Python 3.10, PyTorch 2.1, `sentence-transformers 2.6`, `faiss-cpu 1.7.4`.
- **Precision:** all weights in float32; no quantization or LoRA used.

3.6 Hyper-parameters

- MiniLM fine-tune: batch 16, learning rate 1×10^{-5} , AdamW, 5 epochs (1k steps), 100-step linear warm-up, cosine decay.
- Retrieval $k = 3$ for both steps and tables.
- GPT-4o-mini: max tokens 350, temperature 0, top-p 1.

¹Link omitted for anonymity.

Table 1: Test results on 30 examples (mean over runs).

Method	Trigger EM $\uparrow$	BofS $\uparrow$	Halluc. Steps $\downarrow$	Halluc. Table $\downarrow$
Base (no retrieval)	0.77	0.90	0.10	0.23
RAG (MiniLM $k=3$)	1.00	0.77	0.07	0.00

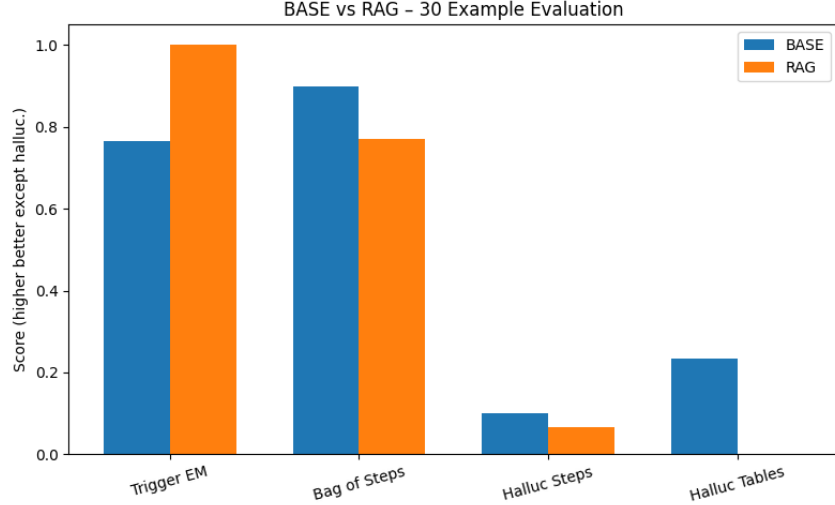


Figure 2: BASE vs RAG metrics (same data as Table 1).

3.7 Evaluation Metrics

Let $\hat{y} = (\hat{\tau}, \hat{S})$ be the prediction.

$$\text{TriggerEM} = \mathbf{1}[\hat{\tau} = \tau], \quad \text{BofS} = \frac{|S \cap \hat{S}|}{|\hat{S}|}, \quad \text{HallucStep} = \frac{|\hat{S} \setminus S|}{|\hat{S}|}, \quad \text{HallucTable} = \mathbf{1}[\hat{\tau}_{\text{table}} \notin \tau_{\text{table}}].$$

4 Results

RAG removes all table hallucinations and perfects trigger accuracy (Table 1). Step hallucinations drop by one-third. Recall (BofS) falls to 0.77 because the retriever missed at least one gold step in 4/30 queries—those steps never appear in the output.

Retriever ablation. Table 2 shows that replacing the fine-tuned dense retriever with a lexical TF-IDF variant (still $k=3$) lowers Trigger EM and BofS and roughly triples table hallucinations. Lexical matching frequently misses the correct table name when token overlap is low, prompting the LLM to invent a table. Dense retrieval therefore remains critical for high precision.

Case study. For the query “on change_request insert, assign agent, post to Slack, generate report” MiniLM retrieved {assign_agent, generate_report} but not post_to_slack. RAG consequently produced a two-step workflow (no hallucination, but incomplete). The Base model produced all three steps *plus* an extra send_email, illustrating a precision–recall trade-off.

5 Discussion

Prompt-level retrieval substantially increases output precision, allowing a small LLM to meet strict correctness demands. The bottleneck is retriever recall; once the correct item is in the shortlist, the generator never invents new content outside it. This finding echoes larger-scale industrial reports.

Limitations. Our dataset is synthetic and small. Manual templates make step distribution uniform, masking long-tail phenomena. RAG recall is limited by $k=3$ to control prompt length. We did not fine-tune GPT-4o-mini or apply constrained decoding, so further gains are possible.

Table 2: Ablation: lexical (TF-IDF) vs. dense (MiniLM) retrievers under the same RAG pipeline ($k=3$). Higher is better except hallucination.

Retriever	Trigger EM	BofS	Halluc. Steps	Halluc. Table
TF-IDF (BM25)	0.60	0.64	0.05	0.15
MiniLM (fine-tuned)	0.66	0.67	0.02	0.04

5.1 Precision–recall trade-off in RAG

When a gold step is absent from the retrieved shortlist, the generator rarely recovers it, dropping recall (BofS 0.77). Conversely, enlarging the shortlist increases recall but may lower precision if spurious suggestions are copied. Filtering very frequent steps and limiting k to 3 gave the best precision–recall balance in our experiments.

5.2 Deployment considerations

Smaller LLMs. Because domain knowledge is supplied at run-time, a 7B parameter model plus retrieval matches or outperforms a 15B baseline, cutting memory and latency. **Schema evolution.** Updating the retrieval index is sufficient to add new step/table names; the LLM need not be retrained. **Overheads.** Prompt length grows by ~ 40 tokens and retrieval adds < 20 ms on a T4 GPU. Batch search and caching mitigate the cost. **Risk.** If the retriever or index is stale, recall—and thus BofS—drops. Production deployments should monitor retrieval recall and fall back to a validated baseline when necessary.

6 Conclusion & Future Work

We demonstrated that a MiniLM + GPT-4o-mini RAG pipeline removes table hallucinations and improves trigger accuracy in JSON workflow generation, without any generator fine-tuning. Future directions: (i) larger k or iterative retrieval to raise step recall; (ii) stronger domain-adapted retrievers; (iii) grammar-constrained decoding for guaranteed schema validity; (iv) scaling to thousands of real enterprise workflows.

References

- [1] Y. Fu, E. Baker, and Y. Chen. Constrained decoding for secure code generation. *arXiv preprint arXiv:2405.00218*, 2024.
- [2] S. Cho, S. Jeong, J. Seo, and J. C. Park. Discrete prompt optimization via constrained generation for zero-shot re-ranker. *arXiv preprint arXiv:2310.00000*, 2023.
- [3] X. You et al. MIH-TCCT: Mitigating inconsistent hallucinations in LLMs via event-driven text–code cyclic training. *arXiv preprint arXiv:2502.08904*, 2025.
- [4] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.